# Convolutional networks

Last modified: 2026-10-30

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers his module on convolutional networks. The exercises are new.

## 1 Welcome

### 1.1 Plan for this lecture

The DATA 471 notes built convolutional layers[1] by hand, with padding[2], stride[3], pooling[4] and channels[5]. This lecture puts them to work in PyTorch:

1. the shapes that `nn.Conv2d` takes and returns;
2. how far across the image one output of a stack of layers can see;
3. what happens to a convolutional network, and to an MLP, when a digit moves by one pixel;
4. a short history of the architectures built from these layers;
5. reusing a network trained on one task to start another.

### 1.2 The digits as images

```
import copy

import numpy as np
import pandas as pd
import torch
from torch import nn

_ = torch.manual_seed(481)
digits = pd.read_csv("data/optdigits-test.csv", header=None)
pixels = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
x = pixels.reshape(-1, 1, 8, 8)
y = torch.tensor(digits[64].to_numpy())
n_train = 1200
order = torch.randperm(len(x))
x_train, y_train = x[order[:n_train]], y[order[:n_train]]
x_test, y_test = x[order[n_train:]], y[order[n_train:]]
```

[1] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[2] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-padding
[3] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-stride
[4] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-pooling
[5] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel

```
loss_fn = nn.CrossEntropyLoss()


def train(net, xs, ys, n_epochs=10, lr=0.05, batch_size=32, seed=0, params=None):
    # Minibatch SGD with momentum 0.9.
    # `params` picks the tensors to update; by default, all of them.
    if params is None:
        params = net.parameters()
    optimizer = torch.optim.SGD(params, lr=lr, momentum=0.9)
    generator = torch.Generator().manual_seed(seed)
    for epoch in range(n_epochs):
        net.train()
        shuffled = torch.randperm(len(xs), generator=generator)
        for start in range(0, len(xs), batch_size):
            batch = shuffled[start:start + batch_size]
            optimizer.zero_grad()
            loss_fn(net(xs[batch]), ys[batch]).backward()
            optimizer.step()
    return net


def accuracy(net, xs, ys):
    # Fraction of the images in `xs` that `net` classifies correctly.
    net.eval()
    with torch.no_grad():
        return (net(xs).argmax(dim=1) == ys).float().mean().item()


def make_cnn(n_classes=10, seed=1):
    # Two 3-by-3 convolutional layers, 2-by-2 max pooling, then a linear layer.
    torch.manual_seed(seed)
    return nn.Sequential(
        nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(),
        nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Flatten(),
        nn.Linear(16 * 4 * 4, n_classes),
    )


def make_mlp(seed=1):
    # One hidden layer of 64 ReLU units, on the 64 pixels.
    torch.manual_seed(seed)
    return nn.Sequential(nn.Flatten(), nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 10))


def n_params(net):
    return sum(p.numel() for p in net.parameters())


print(x_train.shape, x_test.shape)
```

```
torch.Size([1200, 1, 8, 8]) torch.Size([597, 1, 8, 8])
```

The exercises use the UCI optical digits [1], split as in the feedforward networks chapter, with 1,200 training

images, but keep each image as a grid: a tensor of shape $1 \times 8 \times 8$, for one channel[6] of 8 rows and 8 columns. `train()` runs minibatch stochastic gradient descent[7] with momentum[8] 0.9, learning rate[9] 0.05 and batch size 32. `make_cnn()` builds a small convolutional network, with 3,818 parameters[10]. `make_mlp()` builds an MLP with one hidden layer of 64 ReLU units, with 4,810 parameters.

# 2 Convolutional layers in PyTorch

## 2.1 The shape of the output

**Exercise 2.1** (The shape of a convolution's output)**.** `nn.Conv2d(1, 8, 3, padding=1)` is a convolutional layer[a] with 1 input channel[b], 8 output channels[c], $3 \times 3$ filters[d] and padding[e] 1. What shape is its output for the first 32 training images?

**Hint**

The DATA 471 notes give the output size of a convolution[f].

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel
[d] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-filter
[e] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-padding
[f] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#thm-output-size

Solution

*Solution* 2.1.

```
conv = nn.Conv2d(1, 8, 3, padding=1)
conv_out = conv(x_train[:32])
print(conv_out.shape)
```

```
torch.Size([32, 8, 8, 8])
```

`conv_out` has shape $32 \times 8 \times 8 \times 8$: 32 images, each with 8 channels[a] of 8 rows and 8 columns. With padding[b] 1 and stride[c] 1, each side is $\lfloor (8 + 2 - 3)/1 \rfloor + 1 = 8$ pixels long, the same as the input's. PyTorch orders the dimensions as images, channels, rows and columns [2].

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-padding
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-stride

# 3 What one output can see

## 3.1 Two stacked layers

**Exercise 3.1** (What one output can see)**.** `two_convs` below stacks two `nn.Conv2d(1, 1, 3, padding=1)` layers. Which pixels of an $8 \times 8$ input can change the output in row 4, column 4? Count rows and columns from 0.

[6] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel
[7] https://morrison-lab.github.io/mds/optimization.html#def-gradient-descent
[8] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-momentum
[9] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate
[10] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

```
_ = torch.manual_seed(1)
two_convs = nn.Sequential(
    nn.Conv2d(1, 1, 3, padding=1),
    nn.Conv2d(1, 1, 3, padding=1),
)
print(two_convs)
```

```
Sequential(
  (0): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
)
```

Solution

*Solution* 3.1.

```
blank = torch.zeros(1, 1, 8, 8, requires_grad=True)
two_convs(blank)[0, 0, 4, 4].backward()
reaches = (blank.grad[0, 0] != 0).int()
rows_seen = reaches.any(dim=1).sum().item()
cols_seen = reaches.any(dim=0).sum().item()
print(reaches)
```

```
tensor([[0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0]], dtype=torch.int32)
```

At $(4, 4)$, the second layer's output reads a $3 \times 3$ block of the first layer's output, rows and columns 3 to 5. Each of those 9 outputs of the first layer reads its own $3 \times 3$ block of the input, so together they read rows and columns 2 to 6. The gradient[a] of that one output with respect to the input, printed as 1 where it is not 0, is not 0 on a block of 5 rows by 5 columns. Every other pixel leaves the output unchanged.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

**Definition 3.1** (Receptive field)**.** The **receptive field** of a unit in a convolutional network is the set of input pixels whose values can change the unit's value [3, Sec. 9.2].

**Example 3.1** (Receptive fields in `make_cnn()`)**.**

```
def field_size(layers, row, col):
    # Rows and columns of the input that reach any channel of `layers` at (row, col),
    # over 64 training images; one image can miss a pixel through a ReLU that is 0.
    images = x_train[:64].clone().requires_grad_(True)
    layers(images)[:, :, row, col].sum().backward()
    reaches = (images.grad != 0).any(dim=0)[0]
    return reaches.any(dim=1).sum().item(), reaches.any(dim=0).sum().item()


conv_net = make_cnn()
before_pool = field_size(conv_net[:4], row=4, col=4)
after_pool = field_size(conv_net[:5], row=2, col=2)
print(before_pool, after_pool)
```

```
(5, 5) (6, 6)
```

`make_cnn()` stacks two $3 \times 3$ convolutional layers[a], each followed by a ReLU, then $2 \times 2$ max pooling[b]. A unit of the second convolutional layer[c] has a receptive field of 5 by 5 pixels. A unit after the pooling layer takes the largest of a $2 \times 2$ block of those units. Its receptive field is 6 by 6 pixels.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-pooling
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

Remark (How fast a receptive field grows)

*Remark* 3.1 (How fast a receptive field grows). A $k \times k$ convolutional layer[a] with stride[b] 1 makes each side of the receptive field $k-1$ pixels longer. So $L$ such layers of $3 \times 3$ filters[c] give a receptive field of $(2L+1) \times (2L+1)$ pixels, away from the edges of the image. Solution 3.1 found $L = 2$ layers and a $5 \times 5$ block. Pooling[d] and stride[e] make the receptive field grow faster, since each step of a later layer then moves farther across the input [3, Sec. 9.2].

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-stride
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-filter
[d] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-pooling
[e] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-stride

## 3.2 Small filters

**Exercise 3.2** (Two small filters or one large one)**.** Two stacked $3 \times 3$ convolutional layers[a] and one $5 \times 5$ layer have the same receptive field. Suppose every layer has $C$ input channels[b] and $C$ output channels. Which of the two designs has fewer weights, not counting biases?

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel

Solution

*Solution* 3.2. A $k \times k$ layer from $C$ channels[a] to $C$ channels has one $k \times k \times C$ filter[b] per output channel[c], so $k^2C^2$ weights. Two $3 \times 3$ layers have $2 \times 9C^2 = 18C^2$ weights, and one $5 \times 5$ layer has $25C^2$. So the two small layers have fewer weights, 72% as many. They can also apply an activation function between them, as `make_cnn()` does, so they can compute functions of the patch that one layer cannot. The VGG networks are built from stacks of $3 \times 3$ layers for these two reasons [4, Sec. 2.3].

```
channels = 16
small = nn.Sequential(nn.Conv2d(channels, channels, 3), nn.Conv2d(channels, channels, 3))
large = nn.Conv2d(channels, channels, 5)
small_weights = sum(layer.weight.numel() for layer in small)
large_weights = large.weight.numel()
print(small_weights, large_weights)
```

```
4608 6400
```

For $C = 16$, PyTorch's layers have 4,608 and 6,400 weights.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-filter
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel

# 4 Shifted digits

## 4.1 Shifting the input of a convolution

**Exercise 4.1** (Moving the input of a convolution)**.** `shift_right()` below moves each image one pixel to the right, filling the left column with 0. Apply a convolutional layer[a] to a test digit and to its shifted copy. How does the output for the shifted copy compare with the output for the original?

```
def shift_right(images):
    # Move each image in a batch one pixel right; the left column becomes 0.
    moved = torch.zeros_like(images)
    moved[..., 1:] = images[..., :-1]
    return moved


print(x_test[0, 0, :3], shift_right(x_test[:1])[0, 0, :3], sep="\n")
```

```
tensor([[0.0000, 0.0000, 0.4375, 0.8125, 0.1250, 0.0000, 0.0000, 0.0000],
        [0.0000, 0.6875, 0.9375, 0.7500, 0.8125, 0.0000, 0.0000, 0.0000],
        [0.0000, 0.7500, 0.4375, 0.0000, 1.0000, 0.2500, 0.0000, 0.0000]])
tensor([[0.0000, 0.0000, 0.0000, 0.4375, 0.8125, 0.1250, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.6875, 0.9375, 0.7500, 0.8125, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.7500, 0.4375, 0.0000, 1.0000, 0.2500, 0.0000]])
```

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

Solution

*Solution* 4.1.

```
digit = x_test[:1]
with torch.no_grad():
    out_of_shifted = conv(shift_right(digit))
    shifted_out = shift_right(conv(digit))
gap = (out_of_shifted - shifted_out).abs()
gap_inside = gap[..., 1:].max().item()
gap_first_col = gap[..., 0].max().item()
last_col_blank = digit[..., 7].abs().max().item() == 0
print(gap_inside, gap_first_col)
```

```
0.0 0.33131906390190125
```

`conv(shift_right(digit))` is `conv(digit)` shifted one pixel right. Each output pixel is the same filter[a] applied to the patch around it, so moving every patch moves every output. In columns 1 to 7 of every channel[b] of `conv`, the two agree exactly. In column 0 they differ by up to 0.33. There `shift_right()` fills the shifted output with 0, while the layer applies its filter and bias to the new blank column. Column 7 agrees only because this digit's last column is blank.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-filter
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-channel

**Definition 4.1** (Translation equivariance)**.** A function $f$ from images to images is **translation equivariant** when shifting its input shifts its output by the same amount: $f(\text{shift}(x)) = \text{shift}(f(x))$ for every image $x$ and every shift [3, Sec. 9.2]. A convolutional layer[a] with stride[b] 1 is translation equivariant, up to what happens at the edges of the image.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-stride

**Example 4.1** (ReLU and max pooling)**.**

```
relu, pool = nn.ReLU(), nn.MaxPool2d(2)
with torch.no_grad():
    maps = conv(x_test[:1])
    relu_gap = (relu(shift_right(maps)) - shift_right(relu(maps))).abs().max().item()
    pooled_shift = pool(shift_right(maps))
    pooled = pool(maps)
pool_shape = list(pooled.shape[2:])
pool_gap = (pooled_shift - pooled).abs().max().item()
pool_shift_gap = (pooled_shift - shift_right(pooled)).abs().max().item()
print(relu_gap, pool_shape, pool_gap, pool_shift_gap)
```

```
0.0 [4, 4] 0.45514941215515137 0.5029639601707458
```

A ReLU acts on each pixel separately. So a ReLU is translation equivariant with no edge effect. On the feature maps[a] of `conv` for a test digit, `relu(shift_right(maps))` and `shift_right(relu(maps))` agree exactly. A $2 \times 2$ max pooling[b] layer is not translation equivariant. Its output is 4 by 4. A shift of one input pixel is half an output pixel, and no whole-pixel shift of the pooled maps matches a half-pixel shift. The pooled maps of the shifted input differ by up to 0.46 from those of the unshifted input, and by up to 0.50 from those maps shifted one pixel right.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-feature-map
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-pooling

## 4.2 A convolutional network and an MLP

**Exercise 4.2** (Shifted test digits)**.** Train `make_cnn()` and `make_mlp()` with `train()`. Which network loses more test accuracy[a] when every test digit is shifted one pixel right?

```
print(make_cnn(), make_mlp(), sep="\n")
```

```
Sequential(
  (0): Conv2d(1, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU()
  (2): Conv2d(8, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU()
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Flatten(start_dim=1, end_dim=-1)
  (6): Linear(in_features=256, out_features=10, bias=True)
)
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=64, out_features=64, bias=True)
  (2): ReLU()
  (3): Linear(in_features=64, out_features=10, bias=True)
)
```

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

Solution

*Solution* 4.2.

```
cnn = train(make_cnn(), x_train, y_train)
mlp = train(make_mlp(), x_train, y_train)
cnn_acc = accuracy(cnn, x_test, y_test)
cnn_shift_acc = accuracy(cnn, shift_right(x_test), y_test)
mlp_acc = accuracy(mlp, x_test, y_test)
mlp_shift_acc = accuracy(mlp, shift_right(x_test), y_test)
print(cnn_acc, cnn_shift_acc, mlp_acc, mlp_shift_acc)
```

```
0.9882746934890747 0.8107202649116516 0.9614740610122681 0.4438861012458801
```

```
# The prose below says the MLP loses more; stop the render if it does not.
assert mlp_acc - mlp_shift_acc > cnn_acc - cnn_shift_acc
```

| Network | Test accuracy[a] | Shifted test accuracy |
|---|---|---|
| Convolutional | 98.8% | 81.1% |
| MLP | 96.1% | 44.4% |

The MLP loses more: 51.8 percentage points, against 17.8 for the convolutional network. The MLP has a separate weight for each pixel. A stroke that moves to a new column meets weights trained on other strokes. The convolutional layers[b] are translation equivariant. They find the same strokes one column over. The pooling[c] layer is not translation equivariant (Example 4.1). The last linear layer has a separate weight for each pooled position, as the MLP has for each pixel.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-pooling

# 5 Architectures

## 5.1 Four landmarks

Remark (Some convolutional networks that changed the field)

*Remark* 5.1 (Some convolutional networks that changed the field).

- LeNet-5 read handwritten digits with three convolutional layers[a], the first two each followed by subsampling, then a fully connected[b] layer and an output layer [5, Sec. II.B].
- AlexNet won the 2012 ImageNet competition with 5 convolutional layers, ReLU activations, dropout and training on GPUs [6, Sec. 3].
- VGG networks stacked $3 \times 3$ convolutional layers, up to 19 weight layers deep [4, Sec. 2.3], for the reasons in Solution 3.2.
- ResNets added residual connections[c] around pairs or triples of convolutional layers, and trained networks of more than 100 layers [7]. The normalization chapter used residual connections around single fully connected layers.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-fully-connected
[c] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection

# 6 Starting from another task

## 6.1 Few labels

**Exercise 6.1** (Twenty labelled images)**.** Suppose we must classify the digits 5 to 9 but have labels for only 20 images of them. `draw_few()` draws 20 such training images at random, and relabels the digits 5 to 9 as classes 0 to 4. Train `make_cnn(n_classes=5)` from its random starting weights on each draw, for 30 epochs[a]. How accurate is it on the test images of the digits 5 to 9, averaged over 10 draws of the 20 images?

```
n_few = 20
n_draws = 10
is_high = y_train >= 5
x_high, y_high = x_train[is_high], y_train[is_high] - 5
x_high_test = x_test[y_test >= 5]
y_high_test = y_test[y_test >= 5] - 5


def draw_few(draw):
    # Draw `n_few` labelled training images of the digits 5 to 9.
    generator = torch.Generator().manual_seed(draw)
    chosen = torch.randperm(len(x_high), generator=generator)[:n_few]
    return x_high[chosen], y_high[chosen]


print(len(x_high), len(x_high_test))
```

```
602 294
```

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

*Solution* 6.1.

```
scratch_accs = []
for draw in range(n_draws):
    x_few, y_few = draw_few(draw)
    net = train(make_cnn(n_classes=5, seed=2), x_few, y_few, n_epochs=30)
    scratch_accs.append(accuracy(net, x_high_test, y_high_test))
scratch_acc = np.mean(scratch_accs)
print(np.round(scratch_accs, 3))
```

```
[0.908 0.82  0.861 0.827 0.728 0.823 0.806 0.837 0.605 0.874]
```

Averaged over the 10 draws, the network classifies 80.9% of the 294 test images correctly. Its accuracy[a] ranges from 60.5% to 90.8%, depending on which 20 images it was given.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

## 6.2 Reusing layers

**Exercise 6.2** (Reusing the convolutional layers)**.**

```
is_low = y_train < 5
n_low = int(is_low.sum())
low_net = train(make_cnn(n_classes=5), x_train[is_low], y_train[is_low])
low_test = y_test < 5
low_acc = accuracy(low_net, x_test[low_test], y_test[low_test])


def new_head(train_all, seed=3):
    # Copy low_net with a new last layer; return it and the parameters to train.
    net = copy.deepcopy(low_net)
    torch.manual_seed(seed)
    net[-1] = nn.Linear(16 * 4 * 4, 5)
    if train_all:
        return net, net.parameters()
    return net, net[-1].parameters()


print(n_low, low_acc)
```

```
598 0.9933993220329285
```

The training set[a] has 598 labelled images of the digits 0 to 4. `low_net` is `make_cnn(n_classes=5)` trained on them. `new_head()` copies `low_net`, replaces its last linear layer with a new one for the digits 5 to 9, and returns the copy with the parameters[b] to train. With `train_all=False`, those are only the new layer's. The convolutional layers[c] keep the weights they learned on the digits 0 to 4. Trained this way on the 20 images of each draw, is the network more accurate than the networks of Solution 6.1?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

Solution

*Solution* 6.2.

```
frozen_accs = []
for draw in range(n_draws):
    x_few, y_few = draw_few(draw)
    net, params = new_head(train_all=False)
    _ = train(net, x_few, y_few, n_epochs=30, params=params)
    frozen_accs.append(accuracy(net, x_high_test, y_high_test))
frozen_acc = np.mean(frozen_accs)
n_wins = sum(f > s for f, s in zip(frozen_accs, scratch_accs))
n_head = n_params(net[-1])
print(np.round(frozen_accs, 3))
```

```
[0.867 0.837 0.857 0.881 0.779 0.864 0.867 0.888 0.752 0.864]
```

```
# The prose below says the reused layers help; stop the render if they do not.
assert frozen_acc > scratch_acc
```

`low_net` classifies 99.3% of the test images of the digits 0 to 4 correctly. With its convolutional layers[a] kept fixed and a new last layer, the network classifies 84.6% of the digits 5 to 9 correctly, averaged over the 10 draws, against 80.9% for the networks trained from random weights. So it is more accurate on average, by 3.7 percentage points, and on 7 of the 10 draws. The new layer has only 1,285 parameters[b] to fit to the 20 images.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

**Definition 6.1** (Transfer learning)**. Transfer learning** uses what a network learned on one task to start learning another task, usually one with far fewer labelled examples [3, Sec. 15.2]. One common form keeps the early layers of a network trained on the first task, and trains a new last layer for the second. Networks trained on ImageNet are often reused this way: their first layers find edge-like patterns that are useful for many kinds of images [8].

**Example 6.1** (Fixing layers with `requires_grad`)**.**

```
reused, _ = new_head(train_all=False)
for param in reused[:-1].parameters():
    _ = param.requires_grad_(False)
n_trainable = sum(p.numel() for p in reused.parameters() if p.requires_grad)
print(n_trainable, n_params(reused))
```

```
1285 2533
```

Solution 6.2 is transfer learning from the digits 0 to 4 to the digits 5 to 9. It kept the convolutional layers[a] fixed by handing the optimizer only the new layer's parameters[b]. Another common way marks the reused parameters with `requires_grad_(False)`. Then backpropagation computes no gradients[c] for them. Of the 2,533 parameters of the copy, 1,285 still require gradients.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

## 6.3 Training every layer

**Exercise 6.3** (Training every layer from the reused weights)**.** `new_head(train_all=True)` returns all the parameters[a] of the copy, so training changes the convolutional layers[b] too, starting from the weights they learned on the digits 0 to 4. Trained this way on the 20 images of each draw, is the network more accurate than with the convolutional layers kept fixed?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

Solution

*Solution* 6.3.

```
tuned_accs = []
for draw in range(n_draws):
    x_few, y_few = draw_few(draw)
    net, params = new_head(train_all=True)
    _ = train(net, x_few, y_few, n_epochs=30, params=params)
    tuned_accs.append(accuracy(net, x_high_test, y_high_test))
tuned_acc = np.mean(tuned_accs)
print(np.round(tuned_accs, 3))
```

```
[0.895 0.871 0.816 0.871 0.782 0.881 0.837 0.864 0.823 0.83 ]
```

```
# The prose below says this beats random starting weights; stop the render if not.
assert tuned_acc > scratch_acc
```

| Starting weights of the convolutional layers[a] | Layers trained | Mean test accuracy[a] |
|---|---|---|
| Random | All | 80.9% |
| Learned on the digits 0 to 4 | Last only | 84.6% |
| Learned on the digits 0 to 4 | All | 84.7% |

Training every layer gives a mean accuracy of 84.7%, 0.1 percentage points above the network with fixed convolutional layers[b]. So training every layer is about as accurate, not more. Both start from the weights learned on the digits 0 to 4, and both beat the networks that start from random weights.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer
[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

**Definition 6.2** (Fine-tuning)**. Fine-tuning** is transfer learning that trains every layer on the second task, starting from the weights learned on the first, rather than keeping the early layers fixed [8].

**Example 6.2** (A smaller learning rate for the reused layers)**.**

```
tuned, _ = new_head(train_all=True)
optimizer = torch.optim.SGD(
    [
        {"params": tuned[:-1].parameters(), "lr": 0.005},
        {"params": tuned[-1].parameters()},
    ],
    lr=0.05,
    momentum=0.9,
)
group_lrs = [group["lr"] for group in optimizer.param_groups]
print(group_lrs)
```

```
[0.005, 0.05]
```

Solution 6.3 ran fine-tuning with one learning rate[a] for every layer. A PyTorch optimizer also takes a list of groups of parameters[b], each with its own options [9]. `optimizer` here would move the reused convolutional layers[c] with learning rate 0.005 and the new last layer with learning rate 0.05.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

## 6.4 Further reading

Read

*Deep Learning*'s chapter 9[a] covers convolutional networks [3]. Dumoulin and Visin [10] work through convolution arithmetic with diagrams. Yosinski et al. [8] measure how well each layer of an ImageNet network transfers to a new task.

[a] https://www.deeplearningbook.org/contents/convnets.html

# 7 References


[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[11].

[2] PyTorch Contributors, "Torch.nn.Conv2d." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International conference on learning representations*, 2015. Available: https://arxiv.org/abs/1409.1556


[11] https://doi.org/10.24432/C50P49

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791[12].

[6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012. Available: https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90[13].

[8] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in neural information processing systems*, 2014. Available: https://arxiv.org/abs/1411.1792

[9] PyTorch Contributors, "Torch.optim." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/optim.html

[10] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning." 2016. Available: https://arxiv.org/abs/1603.07285

[12] https://doi.org/10.1109/5.726791
[13] https://doi.org/10.1109/CVPR.2016.90